

DESIGN DOCUMENT / SURGICAL ROBOTICS SIMULATION

Surgical Physics Bridge: Comprehensive Architecture & Design Document

Real-Time Physics-Based Simulation for Image-Guided Robotic Interventions via 3D Slicer

This document provides an exhaustive breakdown of the system architecture, code-level implementation details, the rationale behind every design choice, a detailed examination of our inspirations from recent scientific literature, a direct comparison with the state-of-the-art *SlicerROS2* module, and a candid assessment of the remaining work necessary to bring this system to clinical maturity.

CS-498P: MAJOR TECHNICAL PROJECT (MTP)

Rishang Yadav

B23173

SUPERVISOR

Dr. Deepak Raina

Surgical Physics Bridge: Architecture & Design

The *Surgical Physics Bridge* (`SlicerSurgicalBridge`) is a high-performance, real-time simulation framework that couples *3D Slicer* with a *PyBullet* physics backend via the *OpenIGTLink* binary protocol—enabling soft-body deformation, haptic force rendering, and virtual fixture enforcement inside a medical imaging UI.

1 Introduction & Motivation

1.1 CONTEXT OF IMAGE-GUIDED ROBOTIC INTERVENTIONS

Over the last three decades, medical robots have been increasingly utilized to augment human capabilities in a clinical environment. Surgical robots offer high dexterity and precision in hard-to-reach anatomical spaces, correct human inaccuracies such as physiological hand tremors, and reduce the invasiveness of complex surgical procedures. Currently, there are several FDA-approved medical robots used in regular practice, including the **da Vinci** robot for laparoscopic surgery, the **Mako** robot for orthopedic surgery, and the **CyberKnife** robot for radiosurgery.

Parallel to the advancements in medical robotics, the application of surgical navigation and image-guided therapy (IGT) has experienced rapid growth and clinical deployment. The use of surgical navigation systems to execute pre-defined plans based on preoperative imaging is now commonplace. Image-guided robotics represents the confluence of these two domains—procedures that require both active imaging and accurate tool placement.

1.2 THE ROLE OF SIMULATION IN SURGICAL ROBOTICS

Developing image-guided robotic systems requires access to flexible, open-source software and hardware that is inherently safe to test. Given the exorbitant costs of commercial robotic systems and the ethical/safety implications of testing on live subjects (or even cadavers), simulation is a mandatory step in the development pipeline. Simulation provides a risk-free environment to prototype control algorithms, assess ergonomic workflows, test situational awareness algorithms, and validate the safety parameters of robotic motions prior to any physical deployment.

1.3 THE LIMITATIONS OF 3D SLICER IN ISOLATION

For image guidance, the open-source medical imaging platform **3D Slicer** is one of the most adopted tools for research and prototyping. It boasts over a million downloads and an active research community. 3D Slicer natively supports volume rendering, image segmentation, and spatial tracking via optical and electromagnetic sensors (through extensions like **SlicerIGT**).

However, 3D Slicer is fundamentally a *static* data visualization and processing platform. It possesses no inherent understanding of physics, collision, or dynamics. If a tracked robotic tool model intersects with an anatomical model in 3D Slicer, the software simply renders them overlapping; there is no physical resistance, no soft-tissue deformation, and no calculation of contact forces. This critical deficiency means that 3D Slicer alone cannot provide the *situational awareness* necessary for safe robotic interventions.

1.4 THE LIMITATIONS OF PHYSICS ENGINES IN ISOLATION

Conversely, robust physics engines like **PyBullet**, **MuJoCo**, or **SOFA** (Simulation Open Framework Architecture) are designed specifically to handle rigid-body dynamics, soft-body deformations, and complex collision detection. However, these engines are entirely divorced from the clinical imaging ecosystem. They do not natively support DICOM parsing, volume rendering of CT/MRI scans, or point-based medical registration algorithms. Building a custom medical visualization suite on top of a physics engine would be a monumental and redundant task when platforms like 3D Slicer already exist.

1.5 PROPOSED SOLUTION: THE SURGICAL PHYSICS BRIDGE

CORE THESIS

The **Surgical Physics Bridge** (`SlicerSurgicalBridge`) solves this dichotomy. It provides a robust, real-time simulation framework that seamlessly couples 3D Slicer with a PyBullet physics backend. By bridging these two platforms, we combine 3D Slicer's unparalleled visualization, registration, and medical image processing capabilities with PyBullet's dynamic, physics-based constraint formulation, soft-body mechanics, and collision detection.

2 System-Level Architectural Decisions

2.1 THE CASE FOR A DECOUPLED CLIENT-SERVER MODEL

A core architectural decision was to physically and logically decouple the physics engine from the visualization environment using a client-server paradigm.

Physics simulations—especially those involving high-resolution finite element methods (FEM) or soft-body position-based dynamics (PBD)—are computationally expensive. If the physics solver were tightly integrated into 3D Slicer's main thread (or even an adjacent

thread sharing the same memory pool without strict safety), the heavy computation would inevitably block the VTK visualization rendering loop, leading to a stuttering, unresponsive user interface.

By offloading the physics engine to a completely standalone Python environment, we ensure that the PyBullet simulation can run at high frequencies (e.g., 1000 Hz) independently, while 3D Slicer can smoothly render the scene at its optimal display rate (typically 50–60 Hz). The two processes communicate exclusively over the network, guaranteeing complete process isolation.

DESIGN RATIONALE

The decoupled model also enables future deployment across separate machines—e.g., a GPU workstation running a computationally intensive physics backend while a clinical display station runs the Slicer client, connected via a local-area network.

2.2 REJECTION OF ROS AND ROS2: THE "ZERO MIDDLEWARE" PHILOSOPHY

In the robotics community, the Robot Operating System (ROS and recently ROS2) is the de facto standard middleware. It provides an extensive ecosystem of drivers, planning libraries (**MoveIt!**), and visualization tools (**RViz**).

However, we explicitly decided **against** using ROS or ROS2 as our communication bridge. This decision was driven by two critical factors:

- **Complexity and Overhead:** ROS introduces a massive software stack dependency. Requiring clinical researchers to configure Linux environments, source ROS workspaces, manage `colcon` build systems, and debug DDS (Data Distribution Service) middleware presents a massive barrier to entry. Our target users are medical imaging researchers, not robotics systems engineers.
- **Data Structure Inefficiency:** ROS topics are primarily designed for standard robotic messages (`sensor_msgs/JointState`, `nav_msgs/Odometry`, `tf2` transforms). While excellent for kinematics, they are incredibly inefficient for streaming high-density, unstructured raw physics data—such as continuously changing arrays of thousands of floating-point mesh vertices representing deforming soft tissue at 50 Hz.

2.3 EMBRACING OPENIGTLINK FOR HIGH-PERFORMANCE BINARY SERIALIZATION

Instead of ROS, we selected **OpenIGTLink** as our fundamental transport layer. OpenIGTLink is an open-source, lightweight network protocol explicitly designed for Image-Guided Therapy. It provides high-performance, low-latency binary serialization over standard TCP/IP sockets.

Crucially, OpenIGTLink supports the `NDArrayMessage` type. This allows our backend to pack multidimensional arrays of high-precision floating-point data (such as deformed mesh vertices or raw force vectors) into a contiguous binary block and blast it across the network with zero serialization overhead. On the 3D Slicer client side, the `SlicerOpenIGTLinkIF` module natively intercepts these binary arrays and makes them immediately available to the MRML scene graph.

PROTOCOL SUMMARY

Server → Client: `TransformMessage` (4×4 matrices per robot link), `NDArrayMessage` (Nx3 deformed vertices as "DeformedAnatomy"), `NDArrayMessage` (Nx7 contact force vectors as "ContactForces").

Client → Server: `StringMessage` (JSON-encoded joint commands as "CMD"), `StringMessage` (JSON-encoded mesh geometry as "VirtualFixture"), `NDArrayMessage` (raw joint angle targets as "q_des").

2.4 SELECTION OF PYBULLET AS THE PHYSICS ENGINE

We utilized **PyBullet** as the standalone physics server. PyBullet was chosen over alternatives like Gazebo, SOFA, or MuJoCo for the following reasons:

- **Python Native:** It offers incredibly stable, performant Python bindings via `pybullet`, allowing us to build the backend server entirely in Python, vastly accelerating development speed compared to a pure C++ physics backend.
- **Soft-Body Support:** PyBullet features experimental but highly functional support for soft-body dynamics using Neo-Hookean material models via `p.loadSoftBody()`. This is an absolute necessity for simulating surgical tasks where organs and tissues deform dynamically upon interaction with surgical tools.
- **Collision Detection Engine:** Its collision detection engine (based on GJK/EPA algorithms from the Bullet Physics library) is fast enough to compute real-time contacts and normal forces even in complex multi-body environments involving both rigid robots and deformable anatomy.
- **URDF Parsing:** PyBullet natively parses the Unified Robot Description Format (URDF), the de facto standard for robot model descriptions in the robotics community, enabling effortless loading of arbitrary robotic arms.

2.5 3D SLICER CLIENT: C++ LOADABLE MODULE

3D Slicer was chosen as the client due to its dominance in the medical research sphere. Developing the client interface as a **C++ Loadable Module** (`SlicerSurgicalBridge`) rather than a Python Scripted Module was a deliberate choice to ensure maximum performance. Handling high-frequency network sockets (50 Hz data streams) and dynamically updating `vtkPolyData` for dense meshes (thousands of vertices per frame) requires the speed and memory control of compiled C++, preventing the UI from freezing.

WHY NOT A SCRIPTED MODULE?

Slicer's Python environment runs on the main application thread. A Python-based OpenIGTLink handler would compete with the VTK render loop for the Global Interpreter Lock (GIL), causing visible frame drops. Our C++ logic leverages `vtkMRMLIGTLConnectorNode`, which processes incoming network data on a dedicated background thread, ensuring complete non-blocking operation.

3 Detailed Implementation: The Physics Backend (Server)

The physics server is entirely contained within the `surgical_physics_bridge/src/` directory. It consists of three primary components: the main server loop (`server.py`), the concrete PyBullet backend (`pybullet_backend.py`), and the abstract backend interface (`backend_interface.py`).

3.1 THE ABSTRACT BACKEND INTERFACE

To ensure the architecture is not permanently locked to PyBullet, we defined a formal abstract base class (`SurgicalSimulationBackend`) in `backend_interface.py`. This contract specifies the exact API any physics engine must implement to plug into our bridge:

- `load_robot(robot_description: str) → bool` — Load a URDF robot model.
- `load_anatomy(collision_mesh, visual_mesh, material_params) → bool` — Load an anatomical model with separate collision and visual meshes and material properties.
- `set_joint_command(q_des, dq_des, tau)` — Send position, velocity, and/or torque commands to robot joints.
- `step(dt: float)` — Advance the physics simulation by `dt` seconds.
- `get_link_poses() → Dict[str, np.ndarray]` — Return 4×4 pose matrices for every link, already converted to the Slicer RAS coordinate frame.
- `get_tool_state() → dict` — Return end-effector pose and velocity.
- `get_contacts() → list` — Return all current contact points and forces.
- `get_deformed_vertices() → Optional[np.ndarray]` — Return Nx3 vertex positions for soft bodies; `None` for rigid-only simulations.
- `set_virtual_fixture(signed_distance_field, policy)` — Pass a safety boundary and reaction policy (e.g., "keep_out") to the engine.

EXTENSIBILITY

This abstraction means a future contributor could write a `MuJoCoBackend` or `SOFABBackend` class implementing the same interface, and the bridge would function identically without modifying any server or client code.

3.2 INITIALIZING THE SIMULATION ENVIRONMENT

The server operates via `server.py`, which instantiates an OpenIGTLink server listening on a specified port (default 18944) and concurrently initializes the `PyBulletBackend`:

```
server = pyigtl.OpenIGTLinkServer(port=args.port)
backend = PyBulletBackend(gui=args.gui)
# Optional: --gui flag opens the native PyBullet visualizer alongside Slicer
```

3.3 THE SIMULATION LOOP AND EXECUTION TIMING

To maintain real-time accuracy, the server runs a continuous loop. The physics engine is stepped at **1 kHz** (`dt = 0.001` seconds), providing high-fidelity collision detection and stable constraint formulation. However, to prevent flooding the network (and Slicer's rendering pipeline), data is published at a downsampled rate of **50 Hz**.

```
dt = 0.001          # 1 kHz physics loop
publish_rate = 50    # Hz, Slicer display rate
publish_interval = 1.0 / publish_rate

while True:
    # 1. Receive control commands from IGTL
    # 2. Step Physics
    backend.step(dt)
    # 3. Publish State at 50 Hz target rate
```

3.4 DATA EXTRACTION AND OPENIGTLINK FORMATTING

At the 50 Hz publish interval, the server extracts three distinct categories of data from the physics engine:

- 1 Kinematics (Robot Link Transforms):** For every link in the loaded URDF robot, the server extracts the 4×4 homogeneous transformation matrix (position + orientation) and publishes each as a separate `pyigtl.TransformMessage`. The device name is set to the link name, allowing Slicer to automatically create or update a corresponding `vtkMRMLLinearTransformNode` for each link.
- 2 Deformation (Soft-Body Vertices):** If a deformable soft body is loaded (e.g., a liver or kidney model using `p.loadSoftBody()`), the server extracts the current Nx3 array of vertex positions and publishes them as a `float32 NDArrayMessage` with the device name "DeformedAnatomy".
- 3 Haptics (Contact Force Vectors):** The server polls PyBullet for collision events using `get_contacts()`. Each contact point provides: the 3D position on the anatomy surface, the 3D surface normal vector, and the scalar normal force magnitude. These are packed into an Nx7 `NDArrayMessage` (`[x, y, z, nx, ny, nz, f]`) named "ContactForces". When no contacts exist, a single zero-force row is sent to clear the visualization.

3.5 COORDINATE SYSTEM CONVERSION

PyBullet uses a standard right-handed coordinate system (X-right, Y-forward, Z-up), while 3D Slicer's MRML scene operates in the RAS (Right-Anterior-Superior) medical coordinate convention. A dedicated translation layer (`convert_pose_pb_to_slicer`) applies the necessary axis remapping and unit scaling to ensure that all published transforms are in Slicer's native frame of reference.

4 Detailed Implementation: The 3D Slicer Plugin (Client)

The client is an advanced C++ Loadable Module located in `SlicerSurgicalBridge/`. It consists of the module registration (`qSlicerSurgicalBridgeModule.cxx`), the widget/UI layer (`qSlicerSurgicalBridgeModuleWidget.cxx, .ui`), and the core logic (`vtkSlicerSurgicalBridgeLogic.cxx`).

4.1 THE MRML SCENE GRAPH AND NODE OBSERVERS

Slicer's core data structure is the **MRML** (Medical Reality Modeling Language) Scene Graph. Our plugin registers observers on the scene for `NodeAddedEvent`, `NodeRemovedEvent`, and `EndBatchProcessEvent`. When the OpenIGTLink connector pushes new data, Slicer creates or updates corresponding MRML nodes. Our logic class intercepts these events and auto-binds to nodes named "DeformedAnatomy" and "ContactForces":

```
void vtkSlicerSurgicalBridgeLogic::OnMRMLSceneNodeAdded(vtkMRMLNode* node) {
    std::string name = node->GetName();
    if (name == "DeformedAnatomy")
        vtkSetAndObserveMRMLNodeMacro(this->DeformedAnatomyNode, node);
    else if (name == "ContactForces")
        vtkSetAndObserveMRMLNodeMacro(this->ContactForcesNode, node);
}
```

4.2 IN-PLACE MEMORY UPDATES FOR DEFORMABLE ANATOMY

A massive technical hurdle was ensuring Slicer could render thousands of moving vertices at 50 Hz. If we simply replaced the `vtkPolyData` on every frame, the memory reallocation and garbage collection would instantly crash Slicer's frame rate.

Instead, when `ProcessMRMLNodesEvents` detects a modification to the `DeformedAnatomyNode`, it reaches directly into the target `vtkMRMLModelNode`'s raw memory buffer. It extracts the underlying `vtkPoints` array and performs an **in-place update** using `SetComponent()`:

```
vtkDataArray* ptsData = points->GetData();
for (vtkIdType i = 0; i < points->GetNumberOfPoints(); ++i) {
    ptsData->SetComponent(i, 0, dataArray->GetValue(i*3 + 0));
    ptsData->SetComponent(i, 1, dataArray->GetValue(i*3 + 1));
    ptsData->SetComponent(i, 2, dataArray->GetValue(i*3 + 2));
}
points->Modified();
polyData->Modified();
```

CRITICAL DESIGN CHOICE

Calling `Modified()` flags the VTK rendering pipeline to re-render the geometry **without allocating new memory**. The topology (faces/cells) of the mesh remains unchanged—only the vertex positions move. This enables perfectly smooth, real-time soft-tissue visualization at 50 Hz without any memory-management overhead.

4.3 VISUALIZING HAPTIC CONTACT FORCES VIA VTKLINE CELLS

When the `ContactForces` array updates, the plugin dynamically constructs VTK lines to represent force vectors. It filters out negligible forces ($f > 1e-4$), calculates an endpoint by scaling the normal vector by the force magnitude, and inserts a `vtkLine` cell:

```
// Scale factor for visualization
double scale = 10.0 * f;
double p2[3] = { p[0]+n[0]*scale, p[1]+n[1]*scale, p[2]+n[2]*scale };

vtkIdType pid1 = points->InsertNextPoint(p);
vtkIdType pid2 = points->InsertNextPoint(p2);

vtkNew<vtkLine> line;
line->GetPointIds()->SetId(0, pid1);
line->GetPointIds()->SetId(1, pid2);
```

The resulting `vtkPolyData` (containing all the force-vector lines) is assigned to a dedicated `vtkMRMLModelNode` selected by the user in the UI (`forceModelSelector`), rendering real-time force arrows directly in Slicer's 3D view.

4.4 CONSTRUCTING AND SERIALIZING VIRTUAL FIXTURES

To create safety boundaries (keep-out zones), the user selects a `vtkMRMLModelNode` in the UI (the `vfModelSelector`). The C++ code iterates over every point and polygon in the `vtkPolyData`, constructing a dense JSON string containing "vertices" and "indices":

```
ss << "{ \"type\": \"virtual_fixture\", \"policy\": \"keep_out\", ";
ss << " \"vertices\": [ ";
for (vtkIdType i = 0; i < points->GetNumberOfPoints(); ++i) {
    points->GetPoint(i, p);
    ss << "[ " << p[0] << ", " << p[1] << ", " << p[2] << " ] ";
}
ss << "], \"indices\": [...] }";
```

This JSON payload is pushed via the `vtkMRMLIGTLConnectorNode`'s outgoing queue as a `StringMessage` named "VirtualFixture". On the server side, the PyBullet backend parses this JSON and spawns a static collision mesh matching the exact clinical geometry, physically blocking the simulated robot from entering the forbidden zone.

4.5 THE WIDGET AND QT UI LAYER

The widget (`qSlicerSurgicalBridgeModuleWidget`) exposes the following interactive elements in 3D Slicer's module panel:

#	UI ELEMENT	FUNCTION
1	Connect Button	Establishes the OpenIGTLink client connection to the physics server (<code>localhost:18944</code>).
2	q_des Line Edit	Accepts a comma-separated list of target joint angles for manual robot control.
3	Send Command Button	Packages the <code>q_des</code> string into a JSON <code>StringMessage</code> and pushes it to the server.
4	Target Model Selector	Selects the <code>vtkMRMLModelNode</code> that receives deformed soft-body vertex updates.
5	Force Model Selector	Selects the <code>vtkMRMLModelNode</code> that displays rendered contact force arrows.
6	VF Model Selector	Selects the <code>vtkMRMLModelNode</code> whose geometry is exported as a Virtual Fixture.
7	Send VF Button	Serializes and transmits the selected model's mesh as a keep-out zone to the physics engine.

5 Inspirations from the Literature

Our architectural philosophy was deeply influenced by the necessity of moving beyond static imaging. Three primary manuscripts shaped our approach:

5.1 SAHU ET AL. — "INTEGRATING 3D SLICER WITH A DYNAMIC SIMULATOR FOR SITUATIONAL AWARE ROBOTIC INTERVENTIONS" (2401.11715V1)

The Core Concept: Sahu et al. highlight the fundamental gap in image-guided robotics: the lack of physics-based situational awareness. They proposed bridging 3D Slicer (for imaging/navigation) with AMBF (Asynchronous Multibody Framework) to simulate scene dynamics. They identified that intelligent image-guided robotic systems necessitate the integration of contextual models that interpret surgical situations and deliver real-time feedback tailored to the ongoing procedure. Their architecture utilized ROS as the middleware between AMBF and 3D Slicer.

Key Takeaway

Sahu et al. demonstrated that coupling Slicer with a dynamic simulator can provide physics-based situational awareness for complex interventions.

Our Adoption & Evolution: We adopted this exact conceptual framework—the necessity of reflecting dynamic interactions back to the Slicer UI is central to our work. However, where Sahu et al. utilized AMBF and ROS, we pivoted to **PyBullet** and **OpenIGTLink**. Our implementation of dynamic contact force arrows directly answers their call for computational feedback generated in a physics engine to be dynamically reflected back to the Slicer interface, providing the exact situational awareness they described. Additionally, our abstract `SurgicalSimulationBackend` interface means the system could theoretically be adapted to use AMBF as well.

5.2 CONNOLLY ET AL. — SLICERROS2 (2509.19076V1 & SENSORS-22-05336-V2)

The Core Concept: Connolly et al. recognized the severe latency and complexity issues inherent in old middleware bridges (like the ROS-IGTL bridge, which required converting every ROS message to an OpenIGTLink message and vice versa). They developed `SlicerROS2` by compiling the ROS2 C++ API (`rclcpp`) directly into 3D Slicer, effectively turning Slicer into a native ROS2 node. They demonstrated powerful use cases, particularly the use of **Virtual Fixtures** in Breast Conserving Surgery (BCS) to prevent tumor breach, where haptic guidance prevents a surgeon's tool from entering an annotated forbidden zone.

Our Adoption & Evolution:

- Virtual Fixtures:** We drew massive inspiration from their BCS virtual fixture application. Our system replicates this functionality by allowing users to define keep-out zones via Slicer models. However, rather than using ROS `Wrench` topics and computing distance margins dynamically, we serialize the exact mesh geometry, send it to the physics engine, and let PyBullet's highly optimized GJK/EPA collision algorithms handle the rigid-body intersection logic.
- Latency vs. Simplicity:** We shared their absolute intolerance for high latency. However, our solution to bypass middleware latency was not to embed ROS natively into Slicer (which introduces complex build-system dependencies on `rclcpp`, `DDS`, and `colcon`), but to reject middleware entirely in favor of OpenIGTLink binary streams, achieving similar low-latency results with vastly reduced system complexity.

Key Takeaway
SlicerROS2 eliminates the ROS-IGTL bridge latency bottleneck. Virtual Fixtures for BCS demonstrate the clinical value of real-time spatial constraints.

6 Architectural Comparison: Surgical Physics Bridge vs. SlicerROS2

Because both our system and SlicerROS2 aim to bridge robotics and 3D Slicer, a direct comparison is warranted.

DIMENSION	SLICERROS2 (CONNOLLY ET AL.)	SURGICAL PHYSICS BRIDGE (OURS)
Middleware	Full ROS2 ecosystem (<code>rclcpp</code> , <code>DDS</code> , <code>colcon</code>)	None — direct OpenIGTLink TCP sockets
Slicer Integration	Slicer compiled as a native ROS2 node	Standard C++ Loadable Module + <code>SlicerOpenIGTLinkIF</code>
Build Complexity	Requires sourcing ROS2 overlay, <code>colcon</code> workspace	Standard <code>cmake</code> against Slicer-SuperBuild only
Backend Dependencies	External ROS-controlled hardware/simulators	<code>pip install pybullet pyigtl</code>
Primary Data Types	Standard ROS msgs (<code>tf2</code> , <code>JointState</code> , <code>Wrench</code>)	Raw binary arrays (<code>NDArrayMessage</code> for vertices & forces)
Soft-Body Support	Not natively addressed	Core feature — Neo-Hookean deformable meshes via PyBullet
Virtual Fixtures	Distance-margin computation with ROS <code>Wrench</code> feedback	Full mesh export to PyBullet; GJK/EPA rigid collision enforcement
Haptic Rendering	ROS topic-based wrench feedback to haptic devices	Visual force arrows in Slicer (physical device routing planned)
Role of Slicer	Visualization proxy for ROS network	Active orchestrator of a dedicated physics server
Target User	Robotics engineers familiar with ROS	Medical imaging researchers with minimal ROS exposure

7 Remaining Work & Future Clinical Paradigms

While the core pipeline for rigid kinematics, soft-body deformation, haptics, and virtual fixtures is functional, the system requires several critical additions before it can be considered a complete clinical prototype. The foundation is laid, but the control and feedback paradigms remain in their infancy.

7.1 HUMAN-IN-THE-LOOP INTERACTIVE CONTROL CRITICAL

The most pressing limitation of the current implementation is the user interface for robot control.

- **Current State:** To move the robot in the simulation, users must currently type a comma-separated array of joint angles into the `q_des` text box in the Slicer UI. This open-loop, text-based control is entirely insufficient for real-time surgical interaction.
- **Inverse Kinematics (IK) Integration:** We must implement an IK solver that bridges the gap between Slicer's 3D UI and the robot's joint space. By allowing a user to drag an interactive 3D markup node (a `vtkMRMLMarkupsFiducialNode`) in Slicer, we can continuously query the 3D position and orientation of that node. This pose must be sent to the Python server, where PyBullet's `calculateInverseKinematics` function can resolve the required joint angles (`q_des`) to reach that pose.
- **External Spatial Tracker Integration:** For true teleoperation, we must utilize OpenIGTLink to subscribe to live data streams from external optical tracking systems (e.g., NDI Polaris) or electromagnetic trackers (e.g., NDI Aurora). This spatial data must be mapped directly to the robot's end-effector target, allowing physical, tracked surgical tools in the real world to drive the simulated robot in real-time.

7.2 BIDIRECTIONAL HAPTIC FEEDBACK MECHANISMS CRITICAL

Visualizing forces is a crucial first step for situational awareness, but it does not substitute physical feeling.

- **Current State:** Contact forces are intercepted from PyBullet and rendered as visual vector arrows in the 3D Slicer view.
- **Physical Force Reflection:** True virtual fixtures require physical impedance. We must route the `ContactForces` array back out of Slicer (or directly from the Python server) into a physical haptic interface device (such as the 3D Systems Touch / Phantom Omni or a Force Dimension Sigma).
- **Admittance vs. Impedance Control:** When the simulated robot hits the Virtual Fixture boundary (the keep-out zone), the collision engine will generate massive normal forces. These forces must be scaled and fed into the control loop of the haptic device using either an impedance control paradigm (where user motion dictates position, and the simulator dictates force) or an admittance control paradigm (where the user exerts force, and the simulator calculates allowed motion). This ensures that when the virtual robot hits a bone, the physical joystick in the surgeon's hand rigidly locks in place.

7.3 AUTOMATED COORDINATE SYSTEM REGISTRATION PLANNED

The true utility of image-guided surgery lies in perfectly aligning the virtual world with the physical patient.

- **Current State:** The Slicer MRML scene (which operates in the Right-Anterior-Superior or RAS coordinate space) and the PyBullet world frame (typically standard XYZ) are currently aligned using a static, hardcoded translation layer (`convert_pose_pb_to_slicer`). This is fragile and assumes the models are spawned exactly at the origin.
- **Point-Based Rigid Registration:** The system requires an automated fiducial registration module. A user must be able to select corresponding fiducial markers on both the simulated anatomy in Slicer and the physical anatomy (or tracking reference frame). The bridge must then dynamically compute the transformation matrix—typically using Horn's quaternion-based method or Singular Value Decomposition (SVD) to minimize Fiducial Registration Error (FRE).
- **Hand-Eye Calibration (The AX = XB Problem):** For tracking a robot's end-effector using an external camera, the system will eventually need to implement hand-eye calibration protocols to calculate the rigid transformation between the robot's base coordinate frame and the optical tracker's coordinate frame, ensuring that the simulated robot and the rendered patient images exist in a unified, mathematically rigorous spatial reality.

7.4 ADVANCED SOFT-BODY MATERIAL MODELS PLANNED

- **Current State:** Soft-body tissue deformation relies on PyBullet's experimental Neo-Hookean model.
- **Future:** Integrate more sophisticated constitutive models (e.g., Mooney-Rivlin, Ogden) or explore coupling with dedicated FEM solvers (e.g., SOFA via our abstract backend interface) for higher-fidelity biomechanical tissue simulation, particularly for procedures involving cutting and suturing topological changes.

8 Conclusion

The Surgical Physics Bridge represents a highly specialized, optimized approach to solving the physics-deficiency inherent in standard medical imaging platforms. By fiercely prioritizing high-speed binary data transfer over standard middleware, and by implementing strict in-place memory management in C++, the system proves that real-time soft-body synchronization and haptic force rendering are viable inside a surgical planning UI at clinically acceptable frame rates.

While significant work remains in human-in-the-loop control, haptic device integration, and automated coordinate registration, the architectural foundation detailed in this document—the abstract backend interface, the zero-middleware OpenIGTLink transport, the VTK in-place vertex update strategy, and the physics-engine-enforced virtual fixtures—provides a robust, extensible platform for the future of situational-aware robotic surgery research.

9 References

- 1 **Sahu et al.**, "Integrating 3D Slicer with a Dynamic Simulator for Situational Aware Robotic Interventions," [arXiv:2401.11715v1](https://arxiv.org/abs/2401.11715v1), 2024.
- 2 **Connolly et al.**, "Bridging 3D Slicer and ROS2 for Image-Guided Robotic Interventions," [arXiv:2509.19076v1](https://arxiv.org/abs/2509.19076v1), 2025.
- 3 **Connolly et al.**, "SlicerROS2: A Research and Development Module for Image-Guided Robotic Interventions," *Sensors*, vol. 22, no. 14, 5336, 2022. [doi:10.3390/s22145336](https://doi.org/10.3390/s22145336)
- 4 **Tokuda et al.**, "OpenIGTLink: An Open Network Protocol for Image-Guided Therapy Environment," *Int. J. Med. Robot. Comput. Assist. Surg.*, vol. 5, no. 4, pp. 423–434, 2009.
- 5 **Fedorov et al.**, "3D Slicer as an Image Computing Platform for the Quantitative Imaging Network," *Magnetic Resonance Imaging*, vol. 30, no. 9, pp. 1323–1341, 2012.
- 6 **Coumans & Bai**, "PyBullet, a Python module for physics simulation for games, robotics and machine learning," pybullet.org, 2016–2021.
- 7 **Ungi et al.**, "Open-source platforms for navigated image-guided interventions," *Med. Image Anal.*, vol. 33, pp. 181–186, 2016.